

CS 101 - Algorithms & Programming I

Spring 2026 - Lab 8

Due: Week of April 27, 2026

*Remember the **honor code** for your programming assignments.*

*For all labs, your solutions must conform to the CS101 style **guidelines**!*

All data and results should be stored in variables (or constants where appropriate) with meaningful names.

The objective of this lab is to understand and apply the concepts of classes, references, and inheritance through an interactive game. Before starting the implementation, you are encouraged to think about how the game elements interact and organize them on paper as classes with clearly defined responsibilities. You will apply the principles of object-oriented programming to structure the "Star Catcher" game using multiple classes that represent different parts of the gameplay.

0. Setup Workspace

Start VSC and open the previously created workspace named `labs_ws`. Now, under the `labs` folder, create a new folder named `lab8`.

In this lab, you are to create Java classes/files (under `labs/lab8` folder) as described below. You are expected to submit the files for your initial solution (`Main`, `GameEngine`, `GameField`, `Entity`, `Basket`, `Star`, and `Drawer`). For the revision, you may submit additional files, adding the "Rev" suffix to each revision file (e.g., `GameFieldRev`). **Please submit the files without compressing them, and ensure that no other or previous lab solutions are included.**

Your code must match the names and types of variables and the interface of the methods specified.

1. Star Catcher Game

In this game, the player controls a basket at the bottom of the screen to catch falling stars. During each round, stars fall from above, and the player moves left or right to catch them. Catching a star earns one point, while missing a star costs one health point. The game ends when the player's health reaches zero.

Starter code is provided on Moodle. Detailed descriptions of each class and their interactions are given below. You are responsible for completing the required functionality in the specified sections of each class.

Entity Class: This class serves as the base (superclass) for all game objects.

- **Data Members:** `int x`, `int y`, `char symbol`
- **Constructor:** `Entity(int x, int y, char symbol)`
- **Methods:**
 - `collidesWith(Entity other)`: Returns true if both entities share the exact same (`x`, `y`) coordinates.
 - `getX()`, `getY()`, `getSymbol()`: Accessor methods.

Basket Class (Extends Entity): Represents the player-controlled basket.

- **Data Members:** `int health`
- **Constructor:** `Basket(int x, int y, int health)` (Sets symbol to 'U')
- **Methods:**
 - `moveLeft()`: Decreases x by 1 (ensure it stays within bounds).
 - `moveRight()`: Increases x by 1 (ensure it stays within bounds).
 - `getHealth(), setHealth(int health)`: Accessor and mutator for health.

Star Class (Extends Entity): Represents the falling stars.

- **Constructor:** `Star(int x, int y)` (Sets symbol to '*')
- **Methods:**
 - `update()`: Moves the star one step downward by increasing its y coordinate.

GameField Class: Manages the active entities and scoring.

- **Data Members:** `Basket basket, List<Star> stars, int score`
- **Constructor:** `GameField(Basket basket, List<Star> stars)` (Initializes score to 0).
- **Methods:**
 - `spawnStar()`: Spawns one `Star` at the top of the grid (y=0) at a random x position.
 - `checkCollisions()`:
 - If a star shares the same coordinates as the basket, remove the star and increase the score.
 - If a star's y coordinate reaches the bottom boundary, remove the star and decrease the basket's health.
 - `getScore()`: Returns the current score.

GameEngine Class: Implements the main logic and loop.

- **Constants:** `WIDTH, HEIGHT, BASKET_HEALTH.`
- **Data Members:** `GameField field, Scanner in, Basket basket, List<Star> stars.`
- **Methods:**
 - `run()`: The main game loop. Displays controls, renders the game, processes input, and ends when health is 0.
 - `update(char command)`:
 - Processes command ('A' for left, 'D' for right).
 - Calls `update()` on all stars.
 - Calls `field.checkCollisions()`.
 - Randomly decides to call `field.spawnStar()` (e.g., 30% chance each turn).

Drawer Class: Handles all visual output (Static methods only).

- **Methods:**
 - `printControls()`: Prints instructions ("A: Left, D: Right, Q: Quit").
 - `render(Basket basket, List<Star> stars, int score)`: Creates a 2D `char[][]` grid. Places the basket and stars. Calls `printField()`. Prints score and health below the grid.
 - `printField(char[][] grid)`: Prints the grid with borders (= for top/bottom, | for sides).

Main Class:

- **Methods:** `main()` - Creates `GameEngine` and calls `run()`.

Important Note on Coordinates:

In 2D arrays and computer graphics, the coordinate system is slightly different from standard math. The origin (0, 0) is located at the top-left corner.

- The 'x' coordinate represents the column (increases from left to right).
- The 'y' coordinate represents the row (increases from top to bottom).

Therefore, stars falling down will have their 'y' coordinates increasing each turn, while the basket at the bottom will have a fixed, high 'y' coordinate (HEIGHT - 1).

3. Gameplay

Sample Output

You can access the sample output in the text file, `sample.txt`.